



Multi-Format RAG Agent

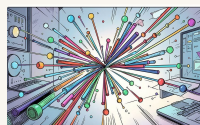
Professional Retrieval-Augmented Generation (RAG) application for document question answering — built with Python, Streamlit, LangChain, FAISS and OpenAI. Upload documents and images, index content, and get answers grounded in source files.

What it Does — At a Glance



Multi-format ingestion

Accepts PDFs, DOCX, PPTX, CSV/Excel and images (PNG/JPG). Each file is parsed with format-specific loaders to preserve page/slide/row context.



Vector indexing

Uses OpenAI embeddings and FAISS to create a semantic vector store for fast similarity search and retrieval.



Grounded answers

Answers are generated only from retrieved chunks. If no relevant content exists the assistant returns: "I couldn't find that in the document."

Security & Credentials

OpenAI API key is validated at runtime and stored only in the Streamlit session. The app never commits keys to source control — production deployments must use environment secrets or a secrets manager.

- ❏ Do not commit API keys. Use environment variables for deployments.



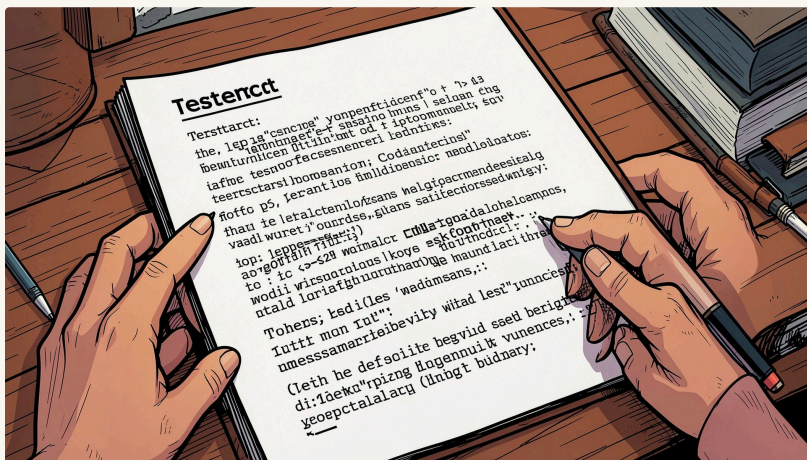
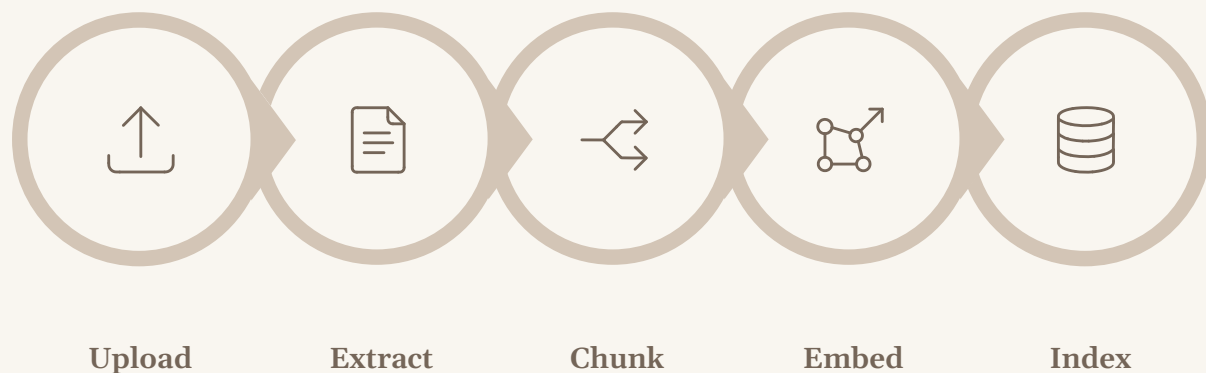


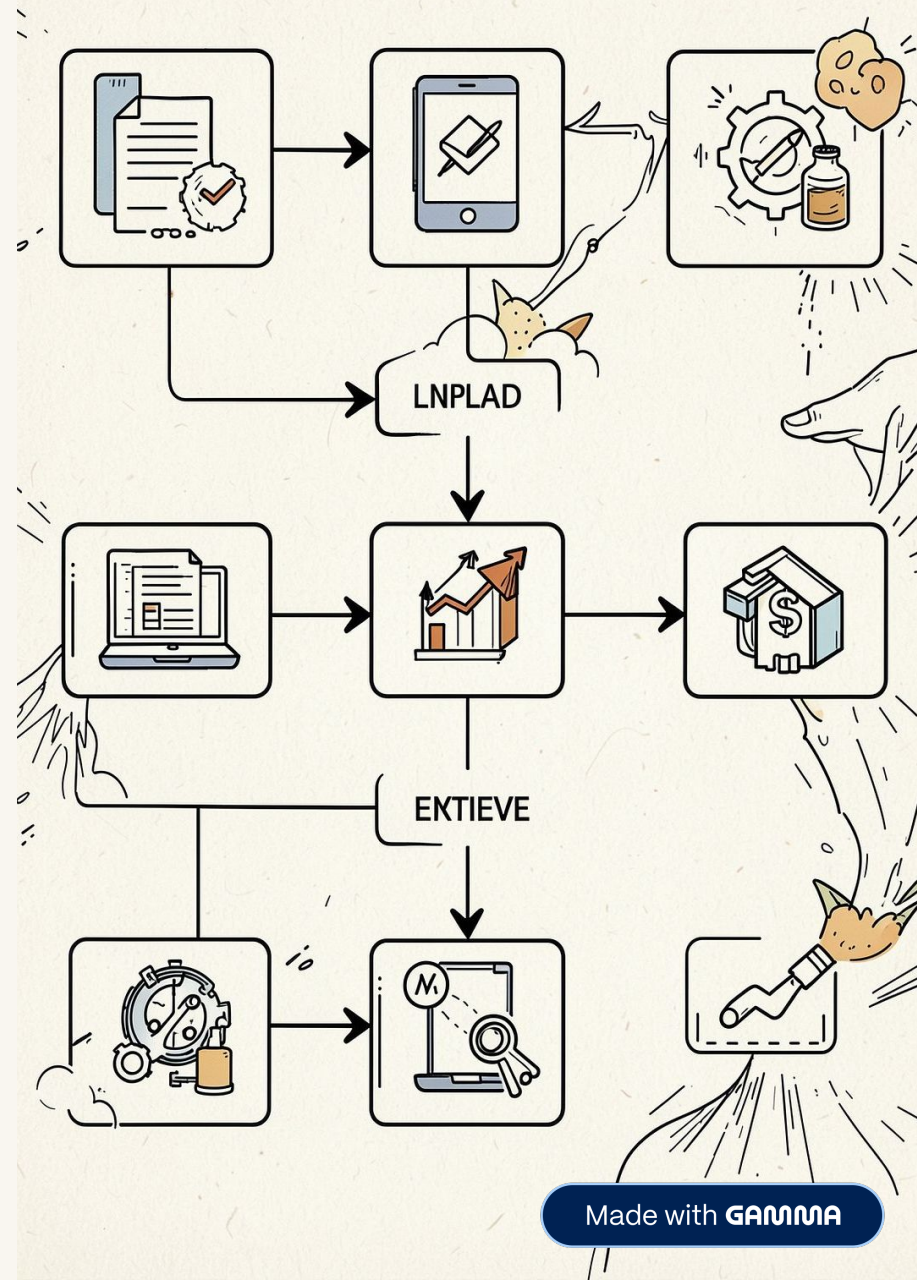
Image OCR & Text Extraction

Image files undergo Tesseract OCR to convert embedded text into searchable content. Scanned PDFs require OCR to expose selectable text for indexing. Extraction preserves page, slide, row or sheet locations to enable precise citations.

RAG Pipeline — How It Works



Each step is configurable (chunk size, overlap, top-k) from the sidebar; changing chunk parameters rebuilds the index for the active chat.





Configurable Retrieval Settings

Model Selection

Choose the OpenAI chat model for generation — trade off cost vs capability.

Chunk Size & Overlap

Controls chunk granularity. Smaller chunks improve precision; overlap preserves context across boundaries.

Top-k Retrieval

Adjust number of retrieved chunks used to compose answers. Higher k increases recall at the cost of latency and token usage.



Source-Grounded Responses

Every generated answer includes source references that show file name and location (page, slide, row, sheet, OCR region). Users can expand retrieved chunks to verify provenance and inspect the exact evidence used.

- i If content isn't found, the assistant returns a clear negative response rather than guessing.

Supported File Types & Processing

- PDF — page-aware text extraction (PyPDF)
- DOCX — paragraph extraction (python-docx)
- PPTX — slide-by-slide extraction (python-pptx)
- CSV / Excel — row and sheet parsing (Pandas, openpyxl)
- Images — OCR via Tesseract (pytesseract, Pillow)



Developer Notes & Project Structure

Repository includes a Streamlit app entrypoint, requirements.txt and README. Key packages: streamlit, langchain, openai, faiss-cpu, pypdf, python-docx, python-pptx, pandas, pillow, pytesseract.

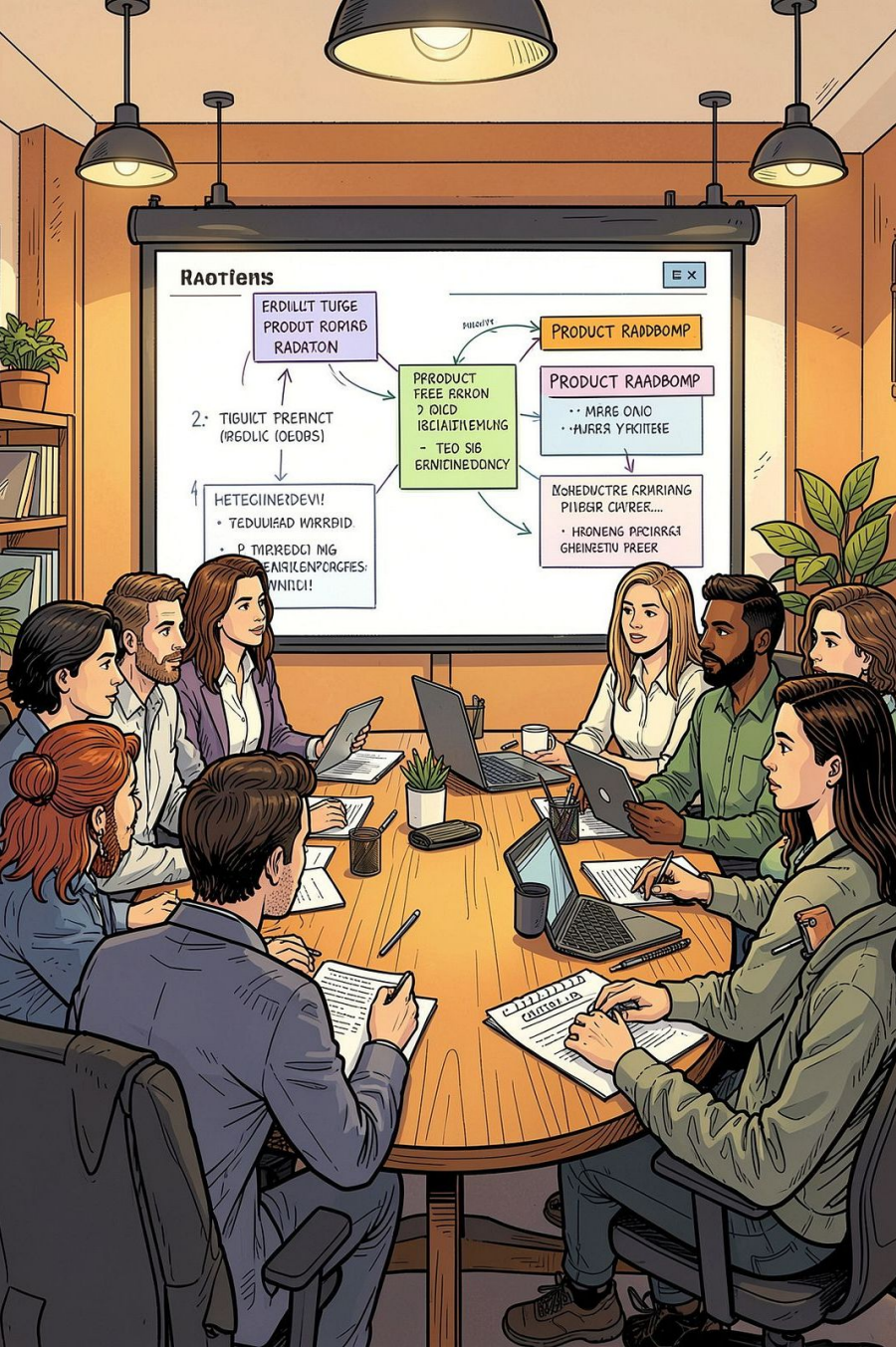
Run locally

```
python -m venv .venv → activate  
→ pip install -r requirements.txt  
→ streamlit run app.py
```

OCR requirement

Install Tesseract on the host for image OCR; optional if you never upload images or scanned PDFs.





Next Steps & Recommendations

- Enable secure secret management for production (secrets manager / env vars).
- Benchmark chunking and top-k settings against representative documents to balance cost and accuracy.
- Instrument logging for retrieval quality (which chunks were used) and user feedback to iteratively improve prompts and pipeline settings.
- Consider hosting FAISS on a persistent backend for multi-user, long-term indexing.

Ready for demo: provide a sample dataset and a short walkthrough to validate retrieval accuracy and citation quality.